



Research Intake 2026

Day 36

3D Vision & Video Processing

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

1 From 2D to 3D: Why Depth Matters	3
2 Camera Models & Projective Geometry	4
3 Epipolar Geometry & Stereo Vision	7
4 Stereo Matching & Disparity Estimation	9
4.1 Disparity-to-Depth Worked Example	10
5 Depth Estimation: Classical to Learned	11
6 Point Clouds & 3D Representations	14
7 Structure from Motion & Visual SLAM	16
8 Video Processing Fundamentals	18
9 Video Diffusion & Neural 3D Representations	20
10 Research Frontiers	22
11 Common Pitfalls: A Practical Debugging Checklist	24
Glossary of Terms	26
Chapter Summary: Key Takeaways	27
Assignment / Practical Project Brief	28
A Core Requirements	28
B Dataset	29
C Deliverables	29
D Grading Rubric	30
E Tips and Common Pitfalls	30
F Submission Checklist	31

The future of AI is bright, and you can be part of it!

1 From 2D to 3D: Why Depth Matters

Every day since Day 26 has operated on a 2D image, or a sequence of 2D images treated frame by frame — a photograph, a video frame, a detected bounding box, a segmentation mask, a generated sample. Even Day 32's tracking, which explicitly reasons across time, never left the image plane: an object's trajectory was a sequence of 2D box positions, and Day 33's segmentation masks were 2D pixel regions however precisely traced. This day introduces the dimension every one of those representations was quietly missing: depth. A photograph is a 2D projection of a 3D world, and this day is about recovering — or, in the generative direction Section 9 develops, synthesizing — the third dimension that projection discards.

This is not an entirely new problem for this compendium. Day 28 §6 introduced the homography — a mathematical description of how a planar surface's appearance changes between two photographs — and flagged, in §6.5, that it was a direct forward reference to this day: homography describes a special, restricted case (a flat scene, or a camera that only rotates), and this day generalizes that same underlying machinery — matched keypoints, RANSAC-based robust fitting — to the genuinely 3D case, where two cameras at different positions observe a fully three-dimensional, non-planar scene. Everything this day builds is, in a precise sense, an extension of content already developed rather than a wholly separate topic bolted onto the module's end.

Why does this extension matter practically? A robot navigating a warehouse needs to know how far away an obstacle is, not merely that an obstacle-shaped region exists somewhere in its camera feed. An augmented-reality headset needs to know exactly where a real table's surface sits in 3D space before it can convincingly place a virtual object on top of it. An autonomous vehicle's perception stack (a recurring application throughout this module, from Day 31 §9.1's detection latency budget through Day 32 §9's tracking discussion) needs to know not just that a pedestrian was detected, but how far away that pedestrian is and how quickly the gap is closing. In every one of these cases, a 2D detection or segmentation result — however accurate in the image plane — is an incomplete answer to the question the application actually needs answered.

Callout Box: *The same ambiguity Day 26 §1.2 first raised in the abstract — vision as an ill-posed inverse problem, since a 2D image is consistent with infinitely many 3D scenes — is the concrete subject of this section. A single photograph of a ball cannot, by itself, distinguish a small ball close to the camera from a large ball far away: both produce an identical circular region of an identical size in the image. Figure 1 makes this literal. Recovering 3D structure from 2D images requires additional information beyond a single image alone — a second camera view (Sections 2–4), a sequence of views from a moving camera (Section 7), or a learned prior about how the world tends to be shaped (Section 5) — and this day is organized around exactly these three sources of additional information, roughly in that order.*

The Single-Image Depth Ambiguity

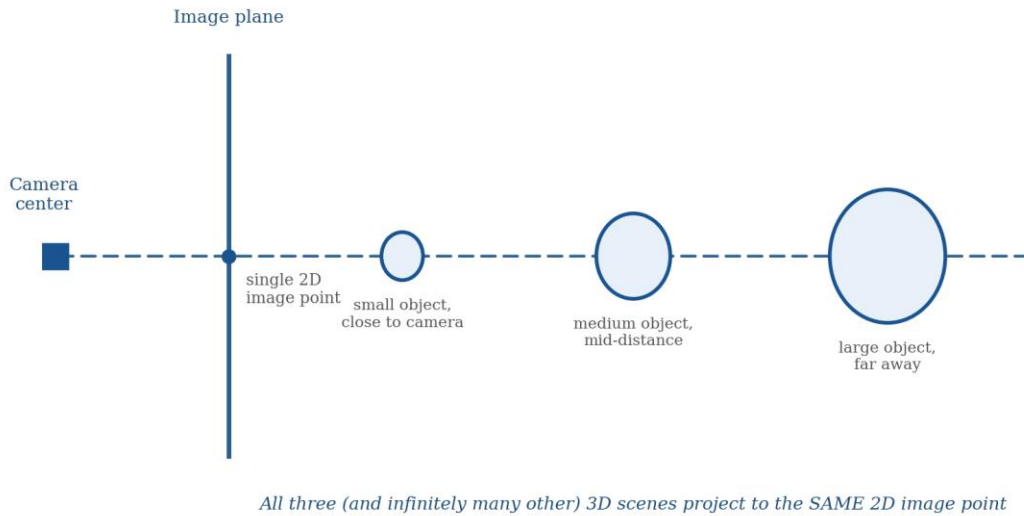


Figure 1. A single 2D image point is consistent with infinitely many 3D scenes at different depths and scales — the same fundamental ambiguity Day 26 §1.2 introduced conceptually, now made geometrically explicit. Everything in Sections 2 through 7 is, in one way or another, a strategy for resolving this ambiguity with additional information.

Section 2 begins with the mathematical model of a single camera — the projection process responsible for the ambiguity in the first place, and the geometric foundation every later section in this day builds on directly.

2 Camera Models & Projective Geometry

Before two cameras can be related to each other (Section 3), or a sequence of camera positions recovered from a moving video (Section 7), it is necessary to understand precisely how a single camera relates the 3D world to its 2D image — the projection process that Section 1 identified as the source of the depth ambiguity in the first place.

2.1 The Pinhole Camera Model

The pinhole camera model is the standard simplified mathematical abstraction of how a camera forms an image: light from a 3D point in the world passes through a single point — the camera center — and lands on a flat image plane behind it, producing a 2D projection. Figure 2 lays out the geometry. A real camera lens is, of course, considerably more complex than an idealized pinhole, but the pinhole model captures the essential projective relationship with enough accuracy for the great majority of computer vision applications, and every other concept in this day — the essential and fundamental matrices of Section 3, the triangulation step of Section 7 — is built directly on top of it.

The Pinhole Camera Model

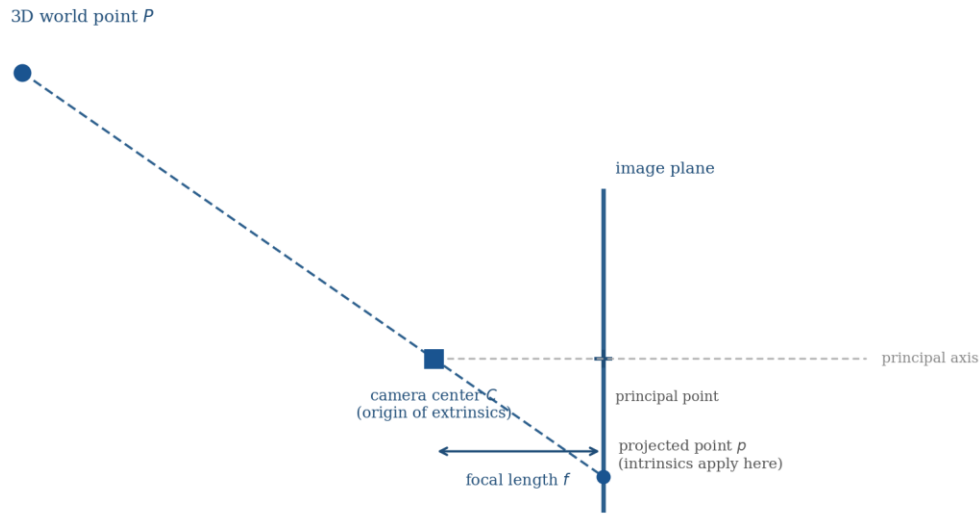


Figure 2. The pinhole camera model: a 3D world point projects through the camera center onto the image plane, at a distance determined by the focal length. Everything to the right of the camera center in this diagram is what a camera's intrinsic parameters (§2.2) describe; everything to the left is what its extrinsic parameters (§2.2) describe.

2.2 Intrinsic and Extrinsic Parameters

A camera's full geometric behavior is captured by two distinct sets of parameters, and keeping the distinction clear is essential for everything that follows. Intrinsic parameters describe the camera's own internal optical properties, independent of where the camera happens to be positioned in the world: the focal length (how strongly the lens bends light, which in turn determines how large a projected object appears), the principal point (where the optical axis actually intersects the image plane, which is not always exactly the image's geometric center), and, in a fuller model, lens distortion coefficients. Extrinsic parameters describe where the camera is and which way it is pointing in the world: a 3×3 rotation matrix and a 3D translation vector relating the camera's own coordinate frame to a shared world coordinate frame.

This distinction matters practically because the two parameter sets are estimated differently and used differently. Intrinsics are usually calibrated once for a given camera and lens combination and then assumed fixed (barring zoom or focus changes); extrinsics change every time the camera moves and must, in general, be re-estimated for every new position — exactly the quantity Section 7's Structure from Motion and visual SLAM content is built to recover continuously as a camera moves through a scene.

2.3 The Camera Projection Matrix

Combining a camera's intrinsics and extrinsics into a single 3×4 projection matrix allows the entire projection process — a 3D world point mapping to a 2D image point — to be expressed as one matrix multiplication in homogeneous coordinates. This compact formulation is what makes the geometric

estimation problems in Sections 3 and 7 tractable: rather than reasoning about lenses and rotations as separate physical processes, the mathematics reduces to linear algebra operations on this single matrix, the same kind of matrix-based reasoning Day 28 §6.2 used to estimate a homography from matched keypoints.

2.4 Lens Distortion and Calibration, at a Conceptual Level

Real lenses, particularly wide-angle lenses, introduce distortion the idealized pinhole model does not account for — straight lines in the world can appear slightly curved in the image, a phenomenon most familiar as the barrel distortion visible in a typical smartphone's ultra-wide camera. Camera calibration is the process of estimating a specific camera's intrinsic parameters (and distortion coefficients) empirically, typically by photographing a known pattern — a checkerboard of precisely known square size is the standard choice — from several different angles and solving for the parameters that best explain the observed pattern of corner positions across those photographs. This calibration step is the direct technical prerequisite for PP17's Path B (uncalibrated stereo, developed in full in the assignment brief at the end of this day), where camera intrinsics are not provided and must be estimated by the student before any depth computation can proceed.



Worked Example: Why Calibration Cannot Be Skipped

Consider two otherwise identical stereo rigs, one using lenses with a 4mm focal length and the other 8mm, photographing the same scene from the same baseline distance apart.

Without knowing which focal length applies, an identical measured pixel disparity between the two images would be interpreted as corresponding to two different real-world depths — the disparity-to-depth relationship developed in Section 4.1 depends directly on focal length as a known quantity, not a free parameter that can be guessed.

This is precisely why Section 4's disparity-based depth computation is only valid once a camera's intrinsics are known — either given directly (PP17 Path A) or estimated through calibration (PP17 Path B) — and why calibration error propagates directly into depth error, a relationship PP17's own comparison between its two paths is designed to make concrete rather than abstract.

3 Epipolar Geometry & Stereo Vision

Section 1 established that a single image cannot, by itself, resolve depth. The most direct fix mirrors how human depth perception works: use two cameras — or one camera photographing the same scene from two positions — and compare the two resulting images. This section develops the geometric relationship between two camera views that makes this comparison tractable, and Section 4 then builds the actual depth-computation algorithm on top of it.

3.1 The Two-Camera Setup

Consider two cameras, each described by the projection machinery Section 2 developed, observing the same 3D scene from two different positions. A single 3D point, visible in both images, projects to one 2D point in the first camera's image and a generally different 2D point in the second camera's image. The core problem stereo vision solves is correspondence: given a point in the first image, find the matching point in the second image that corresponds to the same physical 3D point — precisely the keypoint-matching problem Day 28 §4 developed at length, now applied within a specific, highly constraining geometric setup.

3.2 The Epipolar Constraint

Naively, searching for a match to a point in image one across the entirety of image two is a full 2D search — exactly the brute-force matching cost Day 28 §4.1 quantified. Epipolar geometry reduces this dramatically. Given the two cameras' relative position and orientation, a point in image one does not correspond to just any point in image two — it is mathematically constrained to lie somewhere along a single specific line in image two, called its epipolar line. Figure 3 shows why: the 3D point, together with the two camera centers, defines a plane (the epipolar plane), and this plane intersects each image at a line. Every possible 3D point that could have produced the observed point in image one — the same 'infinitely many depths' ambiguity from Section 1 — projects somewhere along this one line in image two, never anywhere else in the image.

Epipolar Geometry: Reducing Correspondence Search to a Line

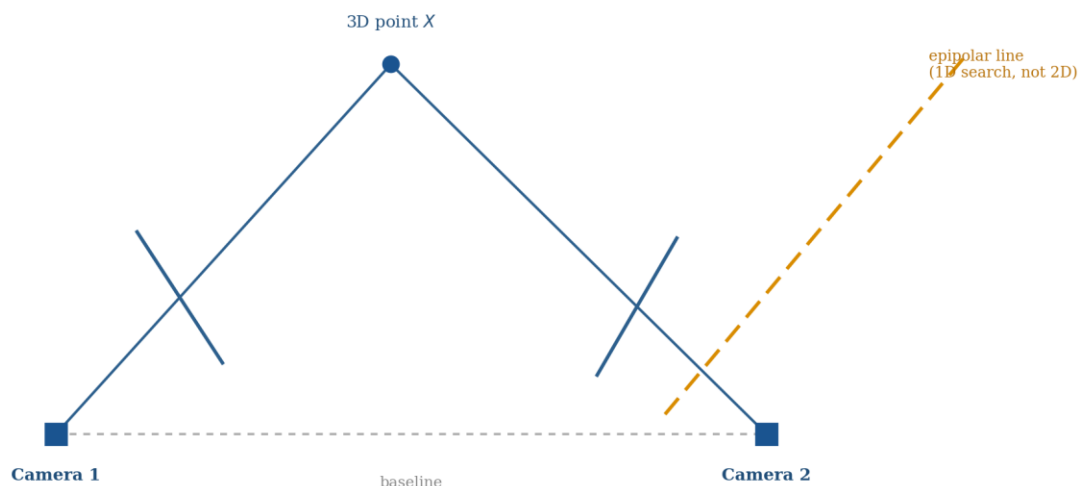


Figure 3. Epipolar geometry: a 3D point together with two camera centers defines a plane, which intersects the second image along a single line (the epipolar line). A correct match for a point in Camera 1's image must lie somewhere along this line in Camera 2's image — reducing correspondence search from a 2D problem to a 1D problem.

This is the direct payoff Day 28's entire feature-matching apparatus was building toward. Once the epipolar geometry between two views is known, finding a correspondence stops being a search across an entire

second image and becomes a search along a single line — a dramatic, structurally guaranteed reduction in search space that makes dense, pixel-by-pixel stereo matching (Section 4) computationally tractable at all.

3.3 Essential and Fundamental Matrices

The epipolar constraint is captured mathematically by a 3×3 matrix relating corresponding points between the two images. When both cameras' intrinsic parameters (Section 2.2) are known, this relationship is called the essential matrix; when intrinsics are not known and only pixel coordinates are available, the more general fundamental matrix captures the same constraint directly in terms of raw pixel correspondences. Both matrices can be estimated directly from a set of matched keypoint pairs between the two images — using exactly the RANSAC-based robust estimation Day 28 §5 developed for homography fitting, with the fundamental or essential matrix taking the place of the homography as the model RANSAC fits. This is not a loose analogy: Day 28 §6.5 named this explicitly as a direct forward reference, and PP17 Path B (developed in full in this day's assignment brief) requires exactly this fundamental-matrix estimation as its first computational step, using the same detect-match-RANSAC pipeline Day 28 built end to end.



Real-World Use Case: Rectification as a Practical Simplification

In practice, once the fundamental or essential matrix between two cameras is known, the two images are almost always warped — a process called stereo rectification — so that corresponding epipolar lines become perfectly horizontal and aligned row by row across both images.

After rectification, the epipolar constraint from Section 3.2 simplifies to its most convenient possible form: a match for a pixel in image one lies somewhere along the exact same horizontal row in image two, never above or below it. This is precisely what makes Section 4's block-matching and cost-volume algorithms efficient to implement, and it is why PP17's calibrated Path A explicitly provides rectified image pairs rather than raw, unrectified stereo photographs.

4 Stereo Matching & Disparity Estimation

With the epipolar constraint reducing correspondence search to a single row of a rectified image pair (Section 3.3), this section develops the actual algorithm for finding those correspondences and converting them into a usable depth measurement — the direct technical core of PP17, this day's practical project.

4.1 Disparity as the Depth Signal

Disparity is the pixel-coordinate difference between where a 3D point appears in the left image and where the same point appears in the right image, after rectification. A point far from the camera shifts only slightly between the two images (small disparity); a point close to the camera shifts substantially more (large disparity) — the same everyday effect of holding a finger close to one eye and alternately closing each eye,

watching the finger appear to jump a large distance against the background. This inverse relationship between disparity and depth is the entire signal stereo vision depends on, and Figure 4 plots it directly.

Disparity-to-Depth: An Inverse Relationship

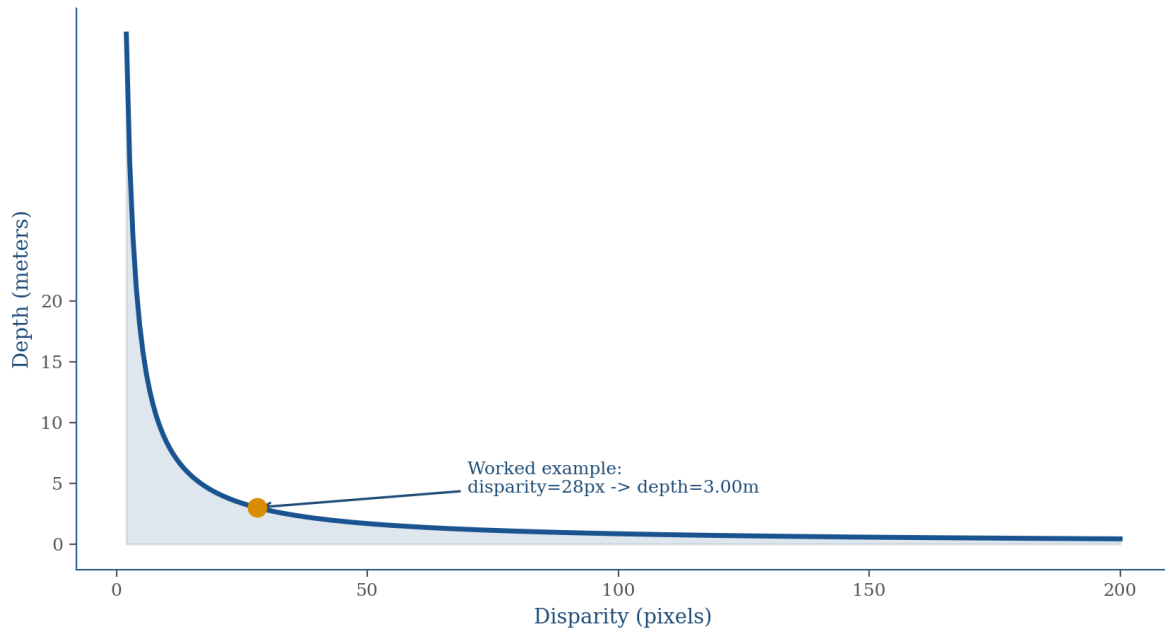


Figure 4. The disparity-to-depth relationship is inversely proportional: $\text{depth} = (\text{baseline} \times \text{focal length}) / \text{disparity}$. Small disparities correspond to large, imprecisely-measured depths (the curve is shallow and flat at low disparity); large disparities correspond to small, precisely-measured depths (the curve is steep). This is why stereo depth estimation is systematically less reliable for distant objects than nearby ones — a point worth remembering when interpreting PPI7's own depth maps.

4.1 Disparity-to-Depth Worked Example

Given a stereo rig with a baseline (the physical distance between the two camera centers) of 0.12 meters and a focal length of 700 pixels, a measured disparity of 28 pixels for a particular point converts to a depth of $(0.12 \times 700) / 28 = 3.00$ meters, using the relationship plotted in Figure 4. Doubling the baseline to 0.24 meters while holding disparity fixed would double the inferred depth to 6.00 meters instead — which is exactly why baseline and focal length must be known precisely (via calibration, Section 2.4) before a measured disparity can be converted into a trustworthy depth value at all: an error in either quantity propagates directly and proportionally into every computed depth.

4.2 Block Matching and Cost-Volume Approaches

The classical algorithm for finding disparity at every pixel is block matching: for each pixel in the left (rectified) image, compare a small window of pixels around it against candidate windows at every possible disparity along the corresponding row of the right image, and select the disparity that produces the best-matching window — using a similarity measure conceptually related to the brute-force descriptor matching Day 28 §4.1 developed, but operating on raw pixel intensities within a small patch rather than a compact

keypoint descriptor. Repeating this per-pixel search across every disparity value under consideration produces what is called a cost volume: a 3D structure of match-quality scores indexed by pixel position and candidate disparity, from which the final disparity map is extracted by selecting, at each pixel, the disparity with the lowest-cost (best) match. Semi-Global Matching (SGM) is the best-known classical refinement of this idea, adding a smoothness constraint that discourages neighboring pixels from being assigned wildly different disparities unless the image data strongly supports it — substantially reducing the noisy, speckled errors raw per-pixel block matching alone tends to produce.

Worked Example: Why Textureless Regions Break Block Matching

Consider a blank, uniformly-painted wall photographed by both cameras in a stereo rig. Every small window of pixels sampled from this wall looks nearly identical to every other window sampled from it — the same lack of local distinctiveness Day 28 §1.1 identified as essential to a good keypoint feature, now causing a direct, predictable failure.

Block matching has no way to distinguish the correct corresponding window from dozens of other, equally plausible candidate windows along the same epipolar row, and the resulting disparity estimate for that region is essentially a guess — unreliable regardless of how carefully the rest of the algorithm is tuned.

This is a property of the scene's texture, not a bug in the matching algorithm, and it is worth checking for directly (per Section 11's debugging checklist) before assuming a poor disparity map in a specific region reflects an implementation error rather than a fundamentally difficult, low-texture input region.

4.3 From Disparity Map to Depth Map

Once a full disparity map has been computed — one disparity value per pixel, wherever a confident match was found — converting it into a depth map is a direct, pixel-wise application of the Section 4.1 worked example's formula: depth equals baseline times focal length, divided by disparity, computed independently at every pixel. Pixels where no confident correspondence was found (commonly in textureless regions, per the callout above, or in regions occluded in one image but not the other) are typically left unfilled or flagged as invalid, rather than assigned an arbitrary depth value — a distinction PP17's evaluation against ground truth needs to handle explicitly, since scoring an intentionally-invalid pixel as a wrong depth prediction would unfairly penalize a disparity estimator for correctly declining to guess.

5 Depth Estimation: Classical to Learned

Section 4's block-matching and cost-volume methods represent the classical approach to stereo depth estimation — and, like every classical technique this compendium has covered since Day 26 §5, they carry real, well-characterized limitations that motivate a learned alternative.

5.1 Limitations of Classical Stereo Matching

Three failure modes recur across classical stereo pipelines. Textureless regions, developed at length in Section 4.2's worked example, leave block matching with no distinctive local signal to match against. Occlusion — a region visible in one camera's image but hidden behind an object from the other camera's viewpoint — has, by definition, no valid correspondence to find at all, since the corresponding 3D surface point simply is not visible in the second image. And repetitive patterns (a tiled floor, a picket fence, a brick wall) produce many equally plausible matches along the same epipolar row, the same fundamentally ambiguous-match problem Day 28 §4.3's Lowe's ratio test was designed to detect and reject, now appearing in the stereo-matching context rather than general keypoint matching.

5.2 Learned Stereo Matching

End-to-end learned stereo matching methods replace the classical, hand-designed cost-volume construction and smoothness regularization with a trained neural network that performs both steps jointly, typically extracting learned features from each image (echoing Day 29's entire treatment of learned versus hand-designed representations), constructing a cost volume from these learned features rather than raw pixel intensities, and applying a 3D convolutional network to regularize and refine the resulting disparity estimate — directly analogous to how Day 29 §6 described the shift from handcrafted classical features to end-to-end learned ones for classification, now recurring for the specific task of stereo correspondence. Learned features are typically more robust to the textureless-region and repetitive-pattern failure modes above, since a network can be trained to exploit broader contextual cues (the fact that a plain wall is usually flat, for instance) rather than relying purely on local pixel-window similarity.

5.3 Monocular Depth Estimation: A Related but Distinct Problem

A genuinely different — and, on first encounter, almost paradoxical — problem is monocular depth estimation: inferring a full depth map from a single image, with no second camera view or stereo pair at all. This appears to directly contradict Section 1's foundational claim that a single image cannot resolve depth. The resolution is that monocular depth estimation does not resolve the ambiguity from raw geometry alone; it resolves it using learned scene priors. A network trained on a very large dataset of images paired with ground-truth depth (collected, in practice, using exactly the stereo or LiDAR-based methods this section otherwise develops) learns strong statistical regularities about how the visual world tends to be structured — that the sky is far away, that a road surface recedes into the distance in a predictable way, that objects resting on a floor are usually supported by it, that a familiar object's apparent size is a strong cue to its distance. These learned priors play exactly the role Day 26 §1.2 identified as necessary in the abstract: additional information beyond the image alone, substituted here for the second camera view Sections 3 and 4 use instead.



Research Spotlight: Monocular Depth Is Learned Regularity, Not Magic

It is worth being precise about what a monocular depth network has actually learned, since the capability can otherwise seem to defy Section 1's own argument. The network has not somehow recovered true 3D geometric information absent from a single 2D image — no such information exists in the raw pixels.

What it has learned is a strong statistical prior over how depth tends to correlate with 2D visual appearance across a very large and diverse training distribution — closely related to how a language model's fluent text generation reflects learned statistical regularities rather than genuine step-by-step reasoning about grammar.

This has a direct, practical consequence: a monocular depth network trained predominantly on outdoor driving scenes will generalize poorly to a cluttered indoor room, and vice versa, since the learned prior is specific to the visual regularities of its training distribution — a domain-generalization concern structurally identical to the one Day 26 §9.2's Gender Shades discussion raised for a very different task.

5.4 A Practical Comparison

Approach	Requires a Stereo Pair?	Fails On	Typical Accuracy
Classical block matching (§4)	Yes	Textureless/repetitive regions, occlusion	Good on well-textured scenes; unreliable elsewhere
Learned stereo matching (§5.2)	Yes	Scenes far outside training distribution	Generally more robust than classical, at higher compute cost
Monocular depth (§5.3)	No	Domains unlike the training distribution	Plausible-looking but less metrically precise than stereo

No single row of this table is a universal winner — a recurring conclusion across this entire compendium's comparison sections (Day 30 §9, Day 31 §8, Day 34 §9.5). Classical stereo matching remains a reasonable, interpretable choice when a calibrated stereo rig is available and real-time performance on modest hardware matters. Learned stereo matching is generally preferable when a stereo rig is available, training data exists, and moderate additional compute is acceptable. Monocular depth is the only option when a stereo pair is genuinely unavailable — a single dashcam, for instance — accepting a real cost in metric precision as the price of that flexibility.

6 Point Clouds & 3D Representations

Once depth has been estimated — by any of Section 5's methods — the result needs to be represented in a form suitable for further processing, storage, or downstream use. This section surveys the spectrum of ways 3D data can actually be represented, since the right choice depends heavily on what happens to the data next.

6.1 The Spectrum of 3D Representations

A point cloud is the most direct representation: simply an unordered set of 3D points, each typically carrying a position and, often, additional attributes like color. A depth map, combined with the camera intrinsics that produced it, converts directly into a point cloud by back-projecting each pixel's depth value into 3D space — the natural, immediate output of Sections 4 and 5's depth estimation. A voxel grid instead organizes 3D space into a regular grid of cubic cells (voxels, the 3D analogue of a 2D pixel), each marked as occupied or empty (or carrying a continuous density value) — a structured representation directly amenable to 3D convolution, at a substantial memory cost that grows cubically with resolution. A mesh represents a surface explicitly as a set of vertices connected into polygonal faces — the standard representation throughout computer graphics and 3D printing, since it directly encodes surface connectivity rather than merely a cloud of disconnected samples. And an implicit or neural representation, previewed briefly here ahead of its full development in Section 9, encodes a 3D shape or scene not as an explicit set of points or vertices at all, but as a continuous function — commonly a small neural network — that can be queried at any 3D coordinate to answer a question like "is this point inside the object" or "what color and density does this point have."

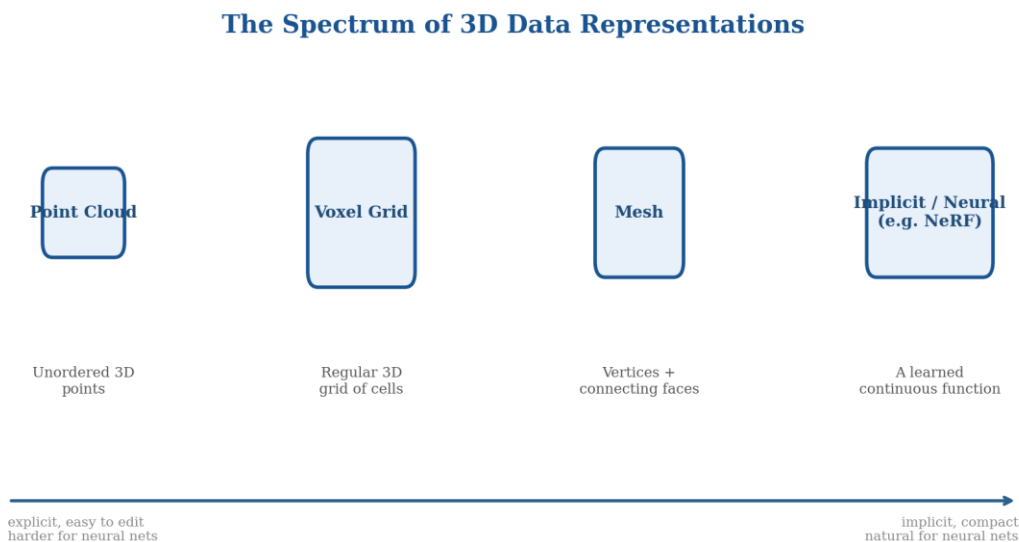


Figure 5. The spectrum of 3D data representations, arranged from explicit and easy to directly edit (point clouds, meshes) to implicit and compact (neural representations). Voxel grids sit in between: explicit and regular, but with a steep memory cost at high resolution. Section 9 develops the rightmost end of this spectrum in full.

6.2 Representation Tradeoffs

Point clouds are memory-efficient relative to a dense voxel grid (only occupied points are stored, not empty space) and are the natural direct output of depth sensing, but carry no explicit surface connectivity — there is no built-in notion of which points belong to the same continuous surface, which complicates tasks like rendering a solid-looking object or measuring surface area. Voxel grids restore a regular grid structure well-suited to adapting standard 2D CNN operations (Day 29 §2) directly into 3D, but memory cost scales with the cube of resolution, making genuinely high-resolution voxel grids impractical for anything beyond fairly

coarse scenes on typical hardware. Meshes are the most storage-efficient representation for smooth surfaces and integrate directly with existing graphics rendering pipelines, but are comparatively difficult to produce directly from noisy sensor data or to process with a standard neural network, since mesh topology (how many vertices, how they connect) is not fixed in the way an image's pixel grid or a voxel grid's cell grid is. Implicit neural representations sidestep the memory-versus-resolution tradeoff voxel grids face — querying the function at arbitrarily fine resolution costs the same regardless of the underlying scene's complexity — at the cost of being comparatively slow to render and requiring a full network evaluation (or many) to extract any explicit geometry at all.

6.3 PointNet: Learning Directly on Point Clouds

PointNet (Qi et al., 2017) addresses a specific architectural puzzle: a point cloud is an unordered set, so a network processing it must produce the same output regardless of the arbitrary order its points happen to be listed in — a property standard convolution, which depends on a fixed spatial grid with well-defined neighbors, does not provide out of the box. PointNet's solution processes each point independently through a shared, identical small network (conceptually similar to how a single convolutional kernel's weights, per Day 29 §2.4, are reused identically at every spatial position), then aggregates the resulting per-point features into one representation using a symmetric function — an operation whose result does not depend on input order, such as taking the maximum value across all points for each feature channel. This combination of shared per-point processing and order-invariant aggregation is what makes PointNet a genuinely foundational architecture for learning directly on point clouds, rather than requiring a costly, information-lossy conversion to a voxel grid first.

Callout Box: *Volumetric and voxel-style 3D representations are not unique to the robotics and graphics applications this section has emphasized so far. Day 37's medical imaging content works with genuinely volumetric data as a matter of course — a CT or MRI scan is a 3D voxel grid of intensity values by its native construction, not an incidental representation choice — meaning the memory-versus-resolution tradeoff developed in Section 6.2 recurs directly, and with real clinical stakes, in that domain.*

7 Structure from Motion & Visual SLAM

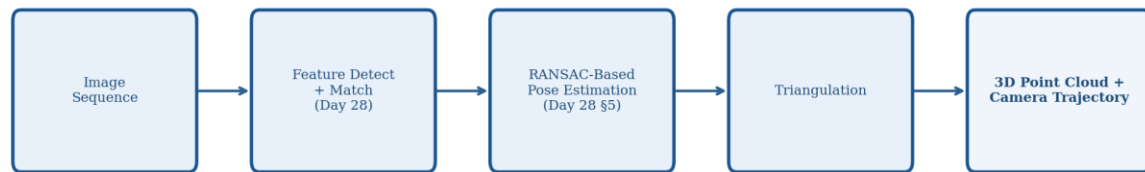
Sections 3 and 4 developed depth estimation from exactly two fixed cameras observing a scene simultaneously. This section extends that same underlying machinery to a substantially more general and more common real-world scenario: a single moving camera, or many cameras at unknown positions, observing a scene across a sequence of images.

7.1 From Two Views to Many

Structure from Motion (SfM) generalizes stereo's two-camera problem to many images — potentially hundreds or thousands — of the same static scene, jointly recovering both the full 3D structure of the scene and the position and orientation of every camera that captured it. This is a direct, large-scale extension of

Section 3's epipolar geometry: rather than estimating one essential or fundamental matrix between a single fixed pair of cameras, SfM estimates relative poses between many overlapping pairs of images and combines them into one globally consistent reconstruction.

The Structure from Motion / Visual SLAM Pipeline



Repeated across many image pairs/frames — the same detect-match-RANSAC machinery from Day 28, applied at scale

Figure 6. The Structure from Motion / visual SLAM pipeline: feature detection and matching (Day 28 §3–§4) between overlapping image pairs, RANSAC-based robust pose estimation (Day 28 §5) to reject outlier matches, triangulation to recover each matched point's 3D position, and accumulation into a consistent 3D point cloud and camera trajectory. This is the same detect-match-RANSAC pipeline Day 28 built end to end, now applied across many image pairs rather than one.

7.2 The SfM Pipeline in Detail

The pipeline shown in Figure 6 proceeds in stages, each of which reuses machinery this compendium has already developed in full. Feature detection and matching between overlapping image pairs uses exactly Day 28 §2–§4's keypoint detectors and descriptors — SIFT's robustness to viewpoint change (Day 28 §3.4.1) is frequently the preferred choice here specifically because SfM datasets often span substantially different viewpoints between images, precisely the condition under which Day 28 flagged SIFT as outperforming faster alternatives like ORB. RANSAC-based robust estimation (Day 28 §5) then fits a relative pose (or fundamental matrix, per Section 3.3 above) between each overlapping image pair, rejecting the inevitable incorrect matches any real feature-matching pipeline produces. Triangulation recovers each matched point's actual 3D position: given a point's 2D location in two or more images together with each image's now-known camera pose, the 3D point is the location that would have produced all of those observed 2D projections simultaneously — the direct geometric inverse of the projection process Section 2.3's camera matrix describes. Finally, bundle adjustment jointly refines every camera pose and every triangulated 3D point simultaneously, minimizing the total reprojection error (the same reprojection-error concept Day 28 §6.2's worked example introduced for a single homography, now computed jointly across an entire multi-image reconstruction) across the whole reconstruction at once.

7.3 Visual SLAM: Structure from Motion in Real Time

Visual SLAM (Simultaneous Localization and Mapping) solves a closely related problem under a much tighter real-time constraint: rather than processing a fixed batch of images offline, as classical SfM typically

does, a robot or handheld device needs to simultaneously build a map of an unfamiliar environment and determine its own position within that map, continuously, as new frames arrive from a live camera feed. Visual SLAM systems accordingly favor incremental, computationally cheap pose estimation for each new frame over global bundle adjustment across the entire history, applying more expensive global optimization only periodically — a direct real-time-versus-accuracy tradeoff structurally similar to Day 31 §8.1's discussion of two-stage versus one-stage detectors, now recurring in the 3D reconstruction context. Loop closure — recognizing that the camera has returned to a previously-visited location, and using that recognition to correct accumulated drift in the estimated trajectory — is a SLAM-specific capability with a direct forward connection to Day 32 §9.2's visual SLAM discussion, which introduced ORB-SLAM as a well-known example built on exactly the ORB descriptor Day 28 §3.3 developed.

Research Spotlight: Learned SLAM and SfM

A growing body of research replaces individual stages of the classical SfM/SLAM pipeline — or, in some cases, the entire pipeline — with learned, end-to-end neural alternatives, echoing the same classical-to-learned progression Section 5 traced for stereo depth estimation specifically.

Learned pose estimation networks can predict relative camera motion directly from a pair of images without explicit feature matching or RANSAC at all, and learned SLAM systems increasingly incorporate the kind of monocular depth priors developed in Section 5.3 to improve robustness in the same textureless or repetitive-pattern conditions that challenge classical stereo matching (Section 5.1).

As with Day 31 §10.1's discussion of DETR removing hand-designed detection machinery, this is not (yet) a wholesale replacement: RANSAC-based robust estimation remains a standard, largely unreplaced component even in otherwise heavily learned SfM and SLAM pipelines, for the same reasons Day 28 §7.3 and §10.3 gave for its continued relevance in learned feature-matching pipelines generally.

The feature-matching and RANSAC content developed here is the direct technical basis for PP17's uncalibrated stereo path — Path B, developed in full in this day's assignment brief — where estimating a fundamental matrix from matched keypoints between an uncalibrated image pair is precisely the two-view special case of the many-view SfM pipeline this section has just developed.

7.4 Connection to Day 32's Tracking

Day 32 developed tracking as maintaining an object's identity consistently across video frames in 2D — a sequence of 2D bounding boxes linked by a consistent track ID. SfM and visual SLAM solve a structurally related consistency problem one dimension up: maintaining a consistent understanding of a scene's 3D structure and the camera's own position across a sequence of frames, rather than an object's 2D position. The two problems are not merely analogous — a complete visual perception system frequently needs both simultaneously, tracking moving objects in 2D image space (Day 32) while separately tracking the observing camera's own motion through 3D space (this section), and a mature autonomous driving or robotics perception stack typically runs both pipelines side by side, sharing much of their underlying feature-detection infrastructure from Day 28.

8 Video Processing Fundamentals

Every section so far in this day has treated a video as a sequence of independent images, related only through the geometric relationships Sections 2–7 developed between camera positions. This section steps back to treat video as its own kind of signal, with processing considerations that apply regardless of whether 3D reconstruction is the eventual goal.

8.1 Video as a Spatiotemporal Signal

A video is naturally understood as a spatiotemporal signal: a function not just of two spatial dimensions (as a single image is) but of two spatial dimensions and one temporal dimension together. This framing directly motivated Day 32 §8.2's three architectural answers to activity recognition — 3D CNNs, which apply convolution across all three dimensions simultaneously; two-stream networks, which separate spatial appearance from temporal motion into two explicit pathways; and skeleton-based approaches, which reduce the spatiotemporal signal to a much lower-dimensional sequence of pose keypoints. Nothing in this section repeats that architectural content; it is worth revisiting Day 32 §8 directly for the modeling side of video understanding. This section instead covers the lower-level signal-processing considerations that apply before any such model is ever trained.

8.2 Optical Flow: A Recap

Day 32 §4.1 developed optical flow in detail — Lucas-Kanade's sparse, per-point tracking and Farneback's dense, per-pixel motion estimation, both built on the brightness-constancy assumption that a pixel's intensity stays roughly constant as it moves between consecutive frames. Optical flow is worth recalling explicitly here because it plays a double role across this compendium: Day 32 used it directly as a tracking and activity-recognition signal (the temporal stream of a two-stream network, per Day 32 §8.2, is typically built directly from dense optical flow), while this day's Section 9 uses temporal coherence — the property optical flow is, in effect, measuring — as a core design requirement for video diffusion models, extending Day 35's image-generation framework across time.

8.3 Temporal Consistency Challenges

Processing a video frame by frame, entirely independently, introduces problems with no single-image analogue. Flicker — small, frame-to-frame variation in a model's output even when the underlying scene content has not meaningfully changed — is the generative-modeling and per-frame-processing analogue of Day 32 §1.1's detection flicker problem: just as an object detector run independently on every frame can produce boxes that flicker in and out of existence, a per-frame image-processing or generation pipeline run independently on every frame can produce visibly inconsistent results (a segmentation boundary that jitters, a colorization that shifts hue) between frames that a human viewer would expect to look essentially identical. Jitter is the closely related visual artifact this inconsistency produces when the affected frames

are played back in sequence — even small, individually reasonable per-frame errors become a distracting, visibly unstable result once assembled into video. Addressing both generally requires explicitly incorporating temporal information into the processing pipeline — Day 32's tracking-based identity persistence and this section's optical-flow-based motion signal are two of the standard tools for doing so, and Section 9's video diffusion models add a third, generative-model-specific mechanism.

8.4 Video-Specific Preprocessing Considerations

Day 27's entire preprocessing treatment — resizing, normalization, augmentation — applies to each individual video frame exactly as it applies to a standalone image, with one important addition specific to video: preprocessing decisions must be applied consistently across every frame of a given clip, not independently re-decided per frame. A random crop augmentation (Day 27 §6.2), for instance, must select the same crop region for every frame within a training clip rather than an independent random crop per frame, or the resulting training signal would contain exactly the kind of artificial frame-to-frame jitter Section 8.3 just described as a failure mode to avoid, rather than a benign augmentation.

8.5 Video Compression Basics Relevant to CV Pipelines

Video files are, in practice, almost never stored as literal sequences of independent full images — doing so would be enormously wasteful, since consecutive frames of real video are usually highly similar to one another. Instead, video codecs exploit this redundancy through a structure of frame types: I-frames (intra-coded, a complete, independently-decodable image, conceptually similar to a standalone JPEG), P-frames (predicted from a preceding frame, storing only the difference), and B-frames (bidirectionally predicted from both preceding and following frames). This matters directly for building a computer vision training data pipeline: decoding an individual frame from the middle of a compressed video, particularly a P-frame or B-frame, is not simply a matter of extracting stored pixel data — it can require decoding a chain of preceding (and sometimes following) frames first, which has a real, sometimes underestimated computational cost when a training pipeline needs to sample individual frames at scale, echoing Day 27 §8.1's broader point about data-loading throughput becoming a training bottleneck if not addressed deliberately.

9 Video Diffusion & Neural 3D Representations

This section brings together two of this day's threads — video (Section 8) and 3D representation (Section 6) — with Day 35's generative modeling content, delivering directly on the forward reference Day 35 §10.2 and §10.3 made explicitly to this day.

9.1 Video Diffusion: Extending Image Generation Across Time

Day 35 developed diffusion models entirely for single, independent images: a forward process gradually adding noise, a learned reverse process removing it, sampled starting from pure noise. Video diffusion extends this same forward-and-reverse noising framework across the temporal dimension, generating a coherent sequence of frames rather than one independent image. The central new challenge is exactly Section 8.3's temporal consistency problem, now in a generative rather than discriminative context: naively applying Day 35's per-image denoising process independently to every frame of a video would reproduce Section 8.3's flicker problem in an especially visible form, since each frame's denoising trajectory would be free to settle on a subtly different, independently-plausible result with no mechanism keeping successive frames visually consistent with one another.

Video diffusion models address this by extending Day 35 §5's U-Net denoising backbone with temporal awareness — commonly through added temporal attention layers (letting a given frame's denoising process directly attend to neighboring frames' current state, a temporal analogue of Day 35 §5.3's spatial self-attention) or 3D convolutional layers that operate across the time dimension jointly with the two spatial dimensions, echoing Day 32 §8.2's 3D CNN approach to activity recognition but applied to generation rather than classification. The practical consequence is that a trained video diffusion model denoises an entire short clip's worth of frames jointly rather than independently, giving the model the structural opportunity to keep them mutually consistent — though achieving this in practice, especially for long clips, remains an active area of ongoing research, developed further in Section 10.2.

9.2 Neural Radiance Fields (NeRF)

Neural Radiance Fields (Mildenhall et al., 2020) instantiate the implicit neural 3D representation Section 6.1 introduced conceptually: rather than storing an explicit point cloud, voxel grid, or mesh, a NeRF represents an entire 3D scene as a small neural network that takes a 3D coordinate and a viewing direction as input and outputs a color and density value at that point — trained end to end from nothing more than a set of ordinary 2D photographs of the scene taken from different, known camera positions (positions recoverable, in practice, using exactly Section 7's Structure from Motion pipeline). Figure 7 shows the resulting capability: once trained on a scene, a NeRF can render a photorealistic image from an entirely new camera viewpoint that was never directly photographed, by querying the learned function along rays cast from that new viewpoint and combining the results through a rendering process closely related to classical volumetric rendering techniques.

Neural Implicit 3D Representation: Multiple Views In, Novel Views Out

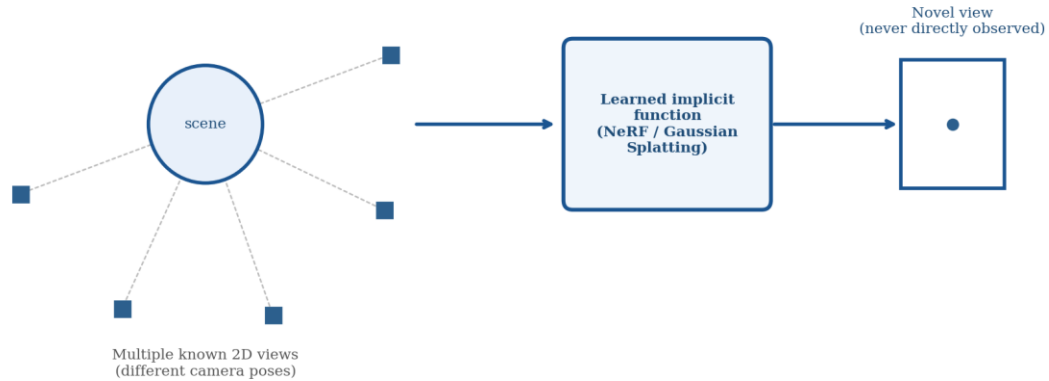


Figure 7. Neural implicit 3D representation: several known 2D views of a scene (with known camera poses, via Section 7's SfM pipeline) train a small neural network to represent the scene as a continuous function. That learned function can then be queried from an entirely new camera viewpoint to synthesize a photorealistic novel view — the practical capability distinguishing NeRF from every explicit representation in Section 6.1.

It is worth being precise about what makes this a genuinely different representation choice from Section 6.1's explicit alternatives, not merely a different file format for the same information. A point cloud or mesh stores a fixed, finite set of geometric primitives; querying it at a location with no stored data returns nothing useful. A NeRF's learned function is defined continuously everywhere in 3D space — it can, in principle, be queried at arbitrarily fine resolution with no increase in stored parameter count, sidestepping the memory-versus-resolution tradeoff Section 6.2 identified as voxel grids' central limitation.

9.3 Gaussian Splatting: A Faster Alternative

A NeRF's central practical drawback is rendering speed: producing a single novel-view image requires many individual network queries along many camera rays, an expensive process that historically made real-time NeRF rendering difficult. 3D Gaussian Splatting (Kerbl et al., 2023) addresses this directly by representing a scene as a large collection of explicit, differentiable 3D Gaussian primitives — each with a learned position, shape, color, and opacity — rather than an implicit neural function. Because these Gaussians are explicit, directly-renderable primitives rather than a function requiring repeated network evaluation, they can be rendered extremely quickly using rasterization techniques closely related to classical computer graphics — trading some of NeRF's representational elegance and continuous queryability for a substantial, practically important rendering-speed advantage, echoing the same explicit-versus-implicit tradeoff Section 6.2 raised in the abstract, now resolved concretely in favor of an explicit representation specifically because rendering speed was the limiting practical constraint.

9.4 Connecting Back to the Representation Spectrum

Representation	Explicit or Implicit?	Rendering Speed	Novel View Synthesis?
Point cloud / mesh (§6.1)	Explicit	Fast	Limited — gaps where no data was captured
Voxel grid (§6.1)	Explicit	Fast, at high memory cost	Limited, resolution-capped
NeRF (§9.2)	Implicit	Slow — many network queries per pixel	Yes — the defining capability
Gaussian Splatting (§9.3)	Explicit	Fast — real-time rendering	Yes, with less compute than NeRF

10 Research Frontiers

10.1 Real-Time Neural 3D Reconstruction

A significant, active research direction pushes Section 9.2–9.3's neural scene representations toward real-time reconstruction — building a NeRF-like or Gaussian-Splatting-like representation of a scene on the fly, from a live camera feed, rather than offline from a fixed, pre-captured set of photographs. This connects directly back to Section 7.3's visual SLAM discussion: where classical visual SLAM builds an explicit point-cloud or sparse-feature map in real time, a growing body of research asks whether a comparably fast pipeline could instead build a rich, photorealistic neural scene representation in real time, combining Section 7's incremental pose-estimation machinery with Section 9's implicit or explicit neural rendering.

10.2 Video Generation Quality and Consistency Challenges

Section 9.1 flagged temporal consistency across long video clips as an unresolved practical challenge, and it remains one of the most actively studied open problems in video diffusion research specifically — maintaining a consistent object identity, a physically plausible motion pattern, and stable fine detail (background texture that should not flicker, faces that should not subtly warp) across dozens or hundreds of generated frames is substantially harder than the single-image consistency Day 35 needed to achieve. Longer generated clips also compound Day 35 §6's sampling-speed drawback directly: generating a coherent multi-second video requires either a very large number of frames processed through Day 35 §6's iterative denoising process, or specialized techniques for generating frames in parallel while preserving the consistency Section 9.1 developed mechanisms for.

10.3 3D-Aware Generative Models

A related and growing research direction combines Day 34's GAN and Day 35's diffusion generative frameworks directly with Section 9's 3D scene representations, producing models capable of generating a 3D-consistent scene or object — one that can be rendered from any viewpoint after generation, rather than

producing a single fixed 2D image the way every generative model through Day 35 did. This directly extends Day 35 §10.3's own forward reference to 3D-aware diffusion, now situated within this day's fuller 3D-representation context (Section 6) rather than mentioned only in passing.

10.4 Forward Reference: Day 37

This day's 3D representation content connects directly and concretely to Day 37's medical imaging focus. Medical imaging modalities — CT and MRI chief among them — are inherently volumetric, three-dimensional data by their native acquisition process, not merely 2D images that happen to be stacked into a 3D-shaped file. Section 6.2's voxel-grid tradeoffs (memory scaling cubically with resolution) apply directly and with genuine clinical weight to a full-resolution CT scan, and Day 37's segmentation and classification architectures for this data build directly on the volumetric representation concepts this day has developed.

10.5 Closing Perspective

This day completed a specific, deliberate arc within the module: Day 26 through Day 35 developed an increasingly sophisticated 2D toolkit — classical features, learned representations, detection, segmentation, tracking, and generation — entirely within the image plane. This day added the dimension every one of those techniques was quietly missing, showing along the way that the addition is not a wholesale departure from earlier content but a direct, geometrically principled extension of it: Day 28's feature matching and RANSAC became this day's Structure from Motion; Day 35's diffusion process became this day's video diffusion; Day 33's segmentation architectures and Day 26's task taxonomy will, in Day 37, meet this day's volumetric representations in medical imaging's highest-stakes application domain. Day 37 closes the Computer Vision Fundamentals module by bringing nearly every earlier day's content — architecture choices, evaluation methodology, and now this day's 3D and volumetric representations — to bear on that single, high-stakes domain, with explainability (a thread running since Day 29 §9's Grad-CAM introduction) as the final, necessary addition.

11 Common Pitfalls: A Practical Debugging Checklist

This day combines geometric estimation, classical matching, and depth computation into a multi-stage pipeline — a poor final depth map or reconstruction can trace its root cause back to any one of several earlier stages. The following checks, worth working through roughly in order, catch the majority of real-world issues.

- **Verify calibration accuracy before debugging disparity estimation.** Per §2.4 and §4.1's worked example, an error in baseline or focal length propagates directly and proportionally into every

computed depth value — always rule this out first, since a perfectly correct disparity map still produces a wrong depth map if the calibration feeding it is wrong.

- **Confirm rectification quality directly, by inspection, before trusting disparity results.** Per §3.3, correct rectification makes corresponding points lie on the exact same image row in both images; overlay a handful of manually-identified corresponding points on both rectified images and check their row alignment directly rather than assuming rectification succeeded.
- **Distinguish a textureless-region failure from a genuine algorithm bug.** Per §5.1's worked example, a blank wall or uniformly-colored surface will produce unreliable disparity regardless of how correctly the matching algorithm is implemented — this is a property of the scene, not a defect to keep debugging.
- **Check whether invalid or unmatched pixels are being silently scored as wrong.** Per §4.3, pixels where no confident correspondence was found should be excluded from quantitative evaluation against ground truth, not treated as an arbitrary incorrect depth prediction.
- **When comparing PP17's Path A and Path B results, separate calibration error from disparity-estimation error explicitly.** A worse Path B result could originate from imprecise checkerboard-based calibration (§2.4), imperfect rectification (§3.3), or genuine disparity-estimation error (§4) — these are three distinct sources with three distinct fixes, and conflating them produces an unhelpful diagnosis.
- **Verify units are consistent throughout the disparity-to-depth computation.** Per §4.1's worked example, baseline is conventionally in meters while focal length and disparity are conventionally in pixels — a silent unit mismatch (millimeters mistaken for meters, for instance) produces a depth map that is wrong by a constant, easily-overlooked scale factor throughout.

A Note on Reproducibility

Structure from Motion and RANSAC-based estimation (§7.2) inherit RANSAC's own randomness, developed at length in Day 28 §5.2 and §11.1 — running the same reconstruction pipeline twice on the same image set can produce a slightly different result, particularly when the inlier ratio is low. Checkerboard-based calibration (§2.4) introduces a further source of run-to-run variation specific to this day: the exact set of checkerboard photographs used, and their specific poses relative to the camera, measurably affects the precision of the resulting calibration. For PP17's comparison between Path A and Path B specifically, fixing random seeds where relevant and, where feasible, running calibration more than once to check its own stability is worth doing explicitly before drawing firm conclusions from a single run.

A Note on Comparing Against Published Benchmarks

Published stereo benchmark results (on datasets such as Middlebury or KITTI, both suggested directly in PP17's dataset guidance below) are typically produced under carefully controlled acquisition conditions,

with precisely known ground truth and, for learned methods, extensive training on large, purpose-built datasets. A PP17 submission's disparity or depth error is very likely to be meaningfully higher than a published leaderboard figure for the same dataset — this is an expected consequence of PP17's necessarily more limited compute and time budget, not by itself evidence of an implementation error, and the meaningful comparison remains PP17's own Path A against its own Path B under matched conditions, exactly as Day 31 §8.3 and every later graded day's assignment brief established as the standard for a fair internal comparison.

Glossary of Terms

This day introduced a substantial amount of specialized geometric and 3D-representation vocabulary. The table below collects the most important terms in one place, each with a pointer back to the section where it was developed in full.

Term	Definition
Pinhole camera model	The idealized geometric model of how a camera projects a 3D scene onto a 2D image plane (§2.1)
Intrinsic parameters	A camera's internal optical properties — focal length, principal point — independent of its position (§2.2)
Extrinsic parameters	A camera's position and orientation in the world — rotation and translation (§2.2)
Camera calibration	Estimating a camera's intrinsic parameters, typically from photographs of a known pattern (§2.4)
Epipolar constraint	The geometric fact that a point's correspondence in a second image must lie along a specific line (§3.2)
Essential / fundamental matrix	A 3×3 matrix encoding the epipolar geometry between two camera views (§3.3)
Stereo rectification	Warping a stereo image pair so corresponding epipolar lines become aligned image rows (§3.3)
Disparity	The pixel-coordinate shift of a point between two rectified stereo images, inversely related to depth (§4.1)
Cost volume	A 3D structure of match-quality scores indexed by pixel position and candidate disparity (§4.2)
Monocular depth estimation	Inferring a depth map from a single image using learned scene priors, no stereo pair required (§5.3)
Point cloud	An unordered set of 3D points, the direct output of back-projecting a depth map (§6.1)
Voxel grid	A regular 3D grid of cells, each marked occupied/empty or carrying a density value (§6.1)
Implicit / neural representation	A 3D scene encoded as a continuous function, queryable at any coordinate (§6.1, §9.2)

Term	Definition
Structure from Motion (SfM)	Jointly recovering 3D scene structure and camera poses from many overlapping images (§7.1)
Bundle adjustment	Jointly refining every camera pose and 3D point to minimize total reprojection error (§7.2)
Visual SLAM	Real-time simultaneous localization and mapping from a live camera feed (§7.3)
Loop closure	Recognizing a previously-visited location to correct accumulated trajectory drift (§7.3)
Optical flow	Per-pixel or per-point estimated motion between consecutive video frames (§8.2)
I-frame / P-frame / B-frame	Video codec frame types trading independence for storage efficiency (§8.5)
Video diffusion	Extending image diffusion models (Day 35) to generate temporally consistent video (§9.1)
Neural Radiance Field (NeRF)	An implicit neural 3D scene representation enabling photorealistic novel-view synthesis (§9.2)
3D Gaussian Splatting	A fast, explicit alternative to NeRF using renderable 3D Gaussian primitives (§9.3)

Chapter Summary: Key Takeaways

- **A single 2D image cannot resolve depth on its own (§1)** — every technique in this day is, in one way or another, a strategy for supplying the additional information a single image lacks: a second camera view, a moving camera's sequence of views, or a learned scene prior.
- **The pinhole camera model's intrinsic and extrinsic parameters (§2)** are the geometric foundation every later section builds on, from epipolar geometry through Structure from Motion.
- **The epipolar constraint reduces stereo correspondence from a 2D search to a 1D search (§3)** — the direct geometric payoff that makes Day 28's feature-matching machinery, and dense stereo matching generally, computationally tractable.
- **Disparity and depth are inversely related through baseline and focal length (§4)** — and this relationship's sensitivity to calibration accuracy is exactly what PP17's Path A vs. Path B comparison is designed to make concrete.
- **Classical stereo matching fails predictably on textureless regions, occlusion, and repetitive patterns (§5.1)**; monocular depth estimation resolves Section 1's ambiguity not through geometry but through learned statistical priors (§5.3).
- **3D data can be represented as point clouds, voxel grids, meshes, or implicit neural functions (§6)**, each trading explicitness, memory cost, and ease of neural processing against one another with no universal winner.

- **Structure from Motion and visual SLAM** extend Day 28's detect-match-RANSAC pipeline across many views (§7), recovering both 3D scene structure and camera trajectory jointly — the direct technical basis for PP17's uncalibrated Path B.
- **Video diffusion (§9.1) and Neural Radiance Fields (§9.2)** extend, respectively, Day 35's generative framework and this day's own representation spectrum into genuinely 3D and temporally-aware generation.

Assignment / Practical Project Brief

PP17: 3D Reconstruction from Stereo Images

★ GRADED ASSIGNMENT

This is a graded assignment, carrying a formal rubric (Section D). The core requirement: reconstruct depth and, optionally, a 3D point cloud from a stereo image pair — first under calibrated, given conditions (Path A), then extended to the genuinely uncalibrated case (Path B), followed by a direct, quantitative comparison between the two. Both paths must be completed, not chosen between; the comparison between them is a required deliverable, not an optional extension.

A Core Requirements

Path A — Calibrated Stereo. Camera intrinsics, extrinsics, and a rectified image pair are provided directly (per §2.2 and §3.3). The task focuses entirely on the core stereo-matching skill developed in Section 4: implement or apply a block-matching or cost-volume disparity estimator (§4.2), convert the resulting disparity map into a depth map using the relationship developed in Section 4.1's worked example, and evaluate the result against provided ground-truth disparity where available. This path deliberately isolates disparity estimation from the added complexity of calibration and rectification, and should be completed first.

Path B — Uncalibrated Stereo. Given only a raw, unrectified pair of images with no camera parameters provided, perform camera calibration yourself using a checkerboard pattern (per §2.4) to estimate intrinsics, then estimate the fundamental matrix from matched keypoints using the detect-match-RANSAC pipeline developed in Day 28 §3, §5, and §7 and applied directly in this day's Section 3.3 and Section 7.2, then rectify the image pair (§3.3) before applying the same disparity-estimation approach used in Path A. This path is considerably more representative of a genuine, real-world 3D reconstruction pipeline end to end, and directly exercises this day's connection back to Day 28's feature-matching content.

B Dataset

A standard stereo benchmark, subsetting appropriately for Colab T4 constraints, is strongly recommended over a self-captured dataset for Path A specifically, since it provides reliable ground-truth disparity for evaluation. The Middlebury Stereo dataset (a small number of scene pairs, with high-quality ground truth, well suited to Path A) or a compact subset of the KITTI stereo benchmark (larger-scale, real driving scenes, somewhat noisier ground truth but a more realistic operating domain) are both reasonable choices. Path B can reuse an unrectified pair from the same benchmark where available, or a self-captured stereo pair together with several checkerboard photographs taken with the same camera — the second option requires more setup effort but produces a more personally meaningful end-to-end result.

Dataset Option	Best Fit	Note
Middlebury Stereo subset	Path A	High-quality ground truth; small enough for a single Colab T4 session
KITTI stereo subset	Path A, or Path B with self-calibration	Larger scale, real-world driving scenes; noisier ground truth
Self-captured pair + checkerboard photos	Path B specifically	Most representative of a genuine pipeline; most setup effort

C Deliverables

- Path A: a computed disparity map and depth map for the provided calibrated, rectified pair, with quantitative evaluation against ground-truth disparity where available (mean/median disparity error is a reasonable default metric).
- Path B: your own estimated camera intrinsics (from checkerboard calibration), your own estimated fundamental matrix and rectification, and the resulting disparity and depth maps computed with the same method used in Path A.
- Optional but encouraged: a 3D point cloud visualization for at least one of the two paths, produced by back-projecting the depth map using the relevant camera intrinsics (§6.1).
- A direct, quantitative comparison between Path A's and Path B's reconstruction quality/error, together with a short written discussion of where the two paths' errors originate — separating calibration accuracy and rectification quality (Path B-specific error sources) from disparity-estimation error itself (a source common to both paths), per §11's debugging checklist.

D Grading Rubric

Rubric Component	Weight	What's Assessed
Path A implementation	25%	Correct disparity/depth computation; sensible evaluation against ground truth

Rubric Component	Weight	What's Assessed
Path B implementation	30%	Correct calibration, fundamental-matrix estimation, and rectification; working end-to-end pipeline
Path A vs. Path B comparison	25%	Rigorous, quantitative comparison; error sources correctly attributed, not merely reported
Code quality & write-up	20%	Clean, documented code; clear written explanation of each pipeline stage

Note the weighting deliberately: Path B's implementation and the comparison between the two paths together account for 55% of the total grade — a majority. PP17 is not primarily an exercise in computing one disparity map (worthwhile, but only 25% on its own); it is primarily an exercise in understanding where 3D reconstruction error actually originates once the convenient simplifications of calibrated, rectified input are removed.

E Tips and Common Pitfalls

- **Rectification errors compound directly into disparity errors** — verify rectification quality by inspection (§11's checklist) before spending time debugging disparity estimation itself.
- **Textureless-region failures are a property of the scene, not a bug** — per §5.1's worked example, expect and document unreliable disparity in flat, uniformly-colored regions rather than treating it as an implementation problem to solve away.
- **Baseline and focal-length units must match throughout** — per §4.1's worked example, a silent unit mismatch produces a depth map that is wrong by a constant scale factor, an easy error to introduce and a hard one to notice without an explicit sanity check against a known real-world distance.
- **Do not conflate calibration error with disparity-estimation error when comparing Path A and Path B** — per §11's checklist, these are distinct sources with distinct fixes, and your written comparison should attribute error to the correct stage.
- **Checkerboard calibration quality depends on having enough varied poses** — photograph the checkerboard from several different angles and distances, not repeatedly from nearly the same position, or the resulting calibration will be poorly constrained.

F Submission Checklist

- Path A: disparity map and depth map computed and evaluated against ground truth where available
- Path B: camera calibration performed and documented (intrinsic reported)

- Path B: fundamental matrix estimated via the Day 28 detect-match-RANSAC pipeline, and rectification applied
- Path B: disparity map and depth map computed using the same method as Path A
- Optional point cloud visualization included for at least one path
- Quantitative comparison between Path A and Path B completed with real measured numbers, not placeholders
- Written discussion explicitly attributes Path B's error to calibration/rectification vs. disparity estimation
- Code is documented, and a short README explains how to reproduce your results

End of Day 36 — ready for gap review and expansion into full compendium.